

# PEARS: Physical-Prior-Guided Efficient Adaptation via Failure Reasoning and Diffusion Steering for Tactile Manipulation

Kun Song<sup>1,\*</sup>, Yiming Wang<sup>2,3,\*</sup>, Yilin Chen<sup>4,\*</sup>, Tianyi Ding<sup>2,\*</sup>,  
Jiaxin Tian<sup>2</sup>, Tianqi Gong<sup>2</sup>, Daolin Ma<sup>2,3,†</sup>, Jia Pan<sup>1,†</sup>

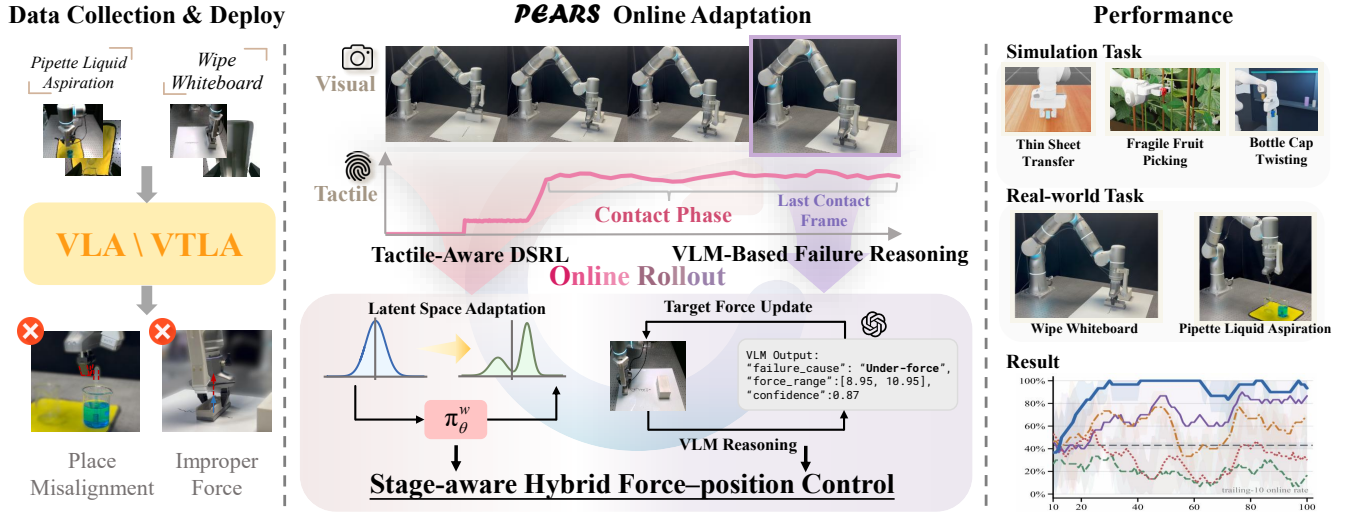


Fig. 1. Overview of PEARS. Tactile-aware DSRL and the physics-guided force reasoning (PFR) module jointly accelerate online adaptation.

**Abstract**—Pretrained robotic policies can suffer substantial performance degradation under out-of-distribution (OOD) conditions encountered during deployment, motivating post-training through real-world interaction. However, reinforcement-learning (RL)-based post-training typically requires substantial environment interactions, a burden that is especially significant in manipulation, where each trial can be slow, costly, or destructive. Therefore, we present PEARS, a physics-prior-guided hybrid RL framework for sample-efficient online adaptation of pretrained policies with tactile feedback. After each episode, its physics-guided force reasoning (PFR) module uses physical priors encoded in a vision-language model (VLM) to diagnose failures from the visual outcome and tactile interaction history and update task-appropriate contact-force bounds. A high-frequency hybrid force-position controller then enforces these bounds during contact. Complementarily, tactile-conditioned diffusion steering reinforcement learning adjusts the latent noise of the frozen flow-matching policy to correct errors in free-space motion and contact timing without updating the base model. In simulation, PEARS improves success rates by 12.4–37.4 percentage points over the strongest per-task baselines. PEARS also reduces the number of interaction episodes required for a certain success threshold by up to 53.2% relative to the fastest baseline. In real-world experiments, PEARS achieves success rates of 95% on Whiteboard Erasing and 90% on Pipette Liquid Aspiration. These results show that combining the PFR module with policy steering can accelerate

adaptation while reducing costly interactions.

## I. INTRODUCTION

Large-scale pretrained vision-language-action (VLA) models [1], [2] have recently emerged as an important paradigm for robotic manipulation, achieving remarkable progress in task generalization across diverse real-world environments. For contact-rich tasks, such as surface wiping and grasping fragile objects, vision-tactile-language-action (VTLA) models [3] extend this paradigm by incorporating tactile feedback to sense and regulate physical interactions with the environment. However, pretrained VLA and VTLA policies can perform poorly during deployment when scene configurations, object properties, or contact dynamics differ from those encountered during training. Such out-of-distribution (OOD) shifts can lead to positioning errors, mistimed contact, or inappropriate interaction forces [4].

Reinforcement learning (RL) post-training with tactile feedback [5], [6] provides a direct route for adapting pretrained policies to the target deployment environment through autonomous interaction. However, traditional RL-based algorithms demand substantial interaction with the environment, which can be time-inefficient and high-cost. For example, a failed online interaction in an industrial operation or chemical laboratory procedure may damage equipment or compromise the experimental setup. This interaction cost is compounded by conventional online RL algorithms, which often rely on stochastic trial-and-error exploration and must

This work was supported by the National Natural Science Foundation of China (NSFC) under Grant Nos. 12522202 and 12272220.

<sup>1</sup>The University of Hong Kong; <sup>2</sup>Shanghai Jiao Tong University; <sup>3</sup>Xense Robotics; <sup>4</sup>University of Pennsylvania.

\*These authors contributed equally to this work. <sup>†</sup>Co-corresponding authors: Daolin Ma and Jia Pan.

visit a sufficiently diverse set of task-relevant states before effective behaviors can be learned. Therefore, how to develop an efficient real-world RL fine-tuning method for contact-rich manipulation with tactile feedback is still an open-problem.

To this end, we introduce **PEARS**, a physics-prior-guided framework for interaction-efficient real-world adaptation in tactile manipulation. Its **physics-guided force reasoning (PFR)** module uses physical priors encoded in a vision–language model (VLM) to reason about episode-level failures and guide contact-force adaptation, rather than relying solely on random RL exploration. After each episode, the PFR module jointly interprets the visual outcome and measured interaction forces to diagnose the failure cause and update task-specific contact-force bounds. During contact, a high-frequency hybrid force–position controller enforces these bounds along the force-control axes while maintaining position tracking in the remaining directions. Complementarily, tactile-conditioned DSRL [7] steers the frozen policy through latent-noise adaptation to correct positioning and contact-timing errors without updating the base model. By separating force adaptation through PFR from latent policy steering, PEARS reduces its reliance on costly real-world trial-and-error exploration.

We evaluated PEARS on three simulated tasks and two real-world manipulation tasks. Experimental results demonstrate that PEARS achieves superior performance, significantly outperforming representative policy adaptation baselines. Our contributions are threefold:

- We present **PEARS**, an interaction-efficient online fine-tuning framework that combines noise-space steering with VLM-based force reasoning using tactile sensing.
- We develop the **PFR module**, which uses physical priors encoded in VLMs to diagnose failure causes and adapt task-specific contact-force bounds, reducing interaction numbers required for policy fine-tuning.
- We conduct extensive experiments across simulated and real-world contact-rich manipulation tasks, demonstrating the effectiveness and efficiency of PEARS.

## II. RELATED WORK

### A. Tactile-Aware Visuomotor Policies

Recent studies have increasingly incorporated tactile sensing into visuomotor policies. These approaches employ tactile encoders [8], [9], [10], [11] cross-modal alignment [12], [13], [14] or auxiliary tactile supervision [15] to leverage learned tactile representations to guide action generation. [16], [17], [3], [18] further pretrain vision–tactile–language–action (VTLA) models on tactile datasets collected across diverse tasks and heterogeneous tactile sensor configurations, enabling VTLA models to learn more unified and transferable tactile representations.

### B. Reactive Tactile Policies for Contact-Rich Manipulation

Contact-rich manipulation requires high-frequency responses to tactile feedback. FOAR [19] uses predicted contact states to dynamically modulate vision-force fusion and

trigger local action corrections. RDP [20] introduces a slow-fast architecture that pairs slow action chunk generation with a fast tactile-conditioned decoder, enabling within-chunk corrections. ForcePolicy [21] and ForceVLA2 [22] employ force feedback to perform high-frequency hybrid force-position control. This reactive design has been extended to tactile-aware VLA models. VLA-Touch [23] uses tactile feedback to support both semantic replanning and action refinement. AT-VLA [24] separates slow vision-language reasoning from fast tactile control. FAVLA [25] adaptively adjusts the action expert invocation frequency based on predicted force variations. UniTacVLA [26] combines predicted and measured tactile signals to apply high-frequency corrections to low-frequency action chunks.

### C. Reinforcement Learning for Tactile Policy Adaptation

Beyond behavior cloning (BC), RL can further improve tactile policies through autonomous interaction with the environment. VLA-RL [27] optimizes autoregressive VLAs through trajectory-level RL. SimpleVLA-RL [28] presents an efficient and scalable RL framework tailored to VLAs.  $\pi_{0.6}^*$  proposes RECAP [29], which trains a value function on demonstrations, autonomous rollouts, and expert interventions. Rather than directly updating VLA parameters, DSRL [7] treats the initial noise of a diffusion or flow-matching policy as the RL latent action space and learns a noise policy, enabling effective autonomous policy improvement in the real world. RL Token [30] compresses task-relevant internal VLA representations into a compact RL token and trains a lightweight actor-critic head on this representation to refine actions while anchoring behavior to the base VLA, enabling sample-efficient online adaptation on real robots. Recent work extends RL-based VLA adaptation to contact-rich manipulation through tactile feedback. Tac-CoRL [5] improves tactile-conditioned VLA policies through simulation-based RL. OmniTacTune [6] learns a tactile residual policy through online RL to provide contact-aware action corrections. TORL-VLA [31] freezes a pretrained tactile-aware VLA during online RL and employs an intervention-censored critic for real-world adaptation. However, most existing RL approaches require substantial environment interaction for stochastic exploration and lack explicit directional guidance, limiting their sample efficiency. To address these limitations, we propose the PEARS framework.

## III. PRELIMINARIES AND PROBLEM FORMULATION

### A. Flow Matching Policy for Imitation Learning

Behavior cloning (BC) learns a policy from a demonstration dataset  $\mathcal{D} = \{(s_t, \mathbf{a}_t)\}$ , where  $s_t \in \mathcal{S}$  is the state and  $\mathbf{a}_t$  is the demonstrated action chunk. For a vision–language–action (VLA) policy,  $s_t$  contains visual observations, proprioception, and the language instruction [32]; a vision–tactile–language–action (VTLA) policy additionally includes tactile observations [3]. In both cases, diffusion and flow-matching policies learn the conditional action distribution by mapping a noise  $\mathbf{w}$  to the demonstrated action distribution.  $\pi_{\text{BC}} : \mathcal{S} \times \mathcal{W} \rightarrow \mathcal{A}$ . At inference time, the policy first

samples an initial noise action and then transforms it into a clean action chunk through an iterative sampling process [7].

### B. Diffusion Steering Reinforcement Learning

A Markov decision process (MDP) can be denoted using  $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ , where  $\mathbf{s}_t \in \mathcal{S}$  is the state,  $\mathbf{a}_t \in \mathcal{A}$  is the physical action, and  $P$ ,  $R$ , and  $\gamma$  denote the transition function, reward function, and discount factor, respectively.

Following DSRL [7], we do not optimize the physical action directly. Instead, the RL policy selects the initial noise of the frozen BC policy. Conditioned on the current state, the noise policy and the resulting action are defined as

$$\mathbf{w}_t \sim \pi_{\theta}^w(\mathbf{w}_t | \mathbf{s}_t), \quad \mathbf{a}_t = \pi_{\text{BC}}(\mathbf{s}_t, \mathbf{w}_t). \quad (1)$$

Under the original full-dimensional noise parameterization, sampling  $\mathbf{w}_t \sim \mathcal{N}(0, \mathbf{I})$  recovers the original BC behavior. Learning  $\pi_{\theta}^w$  instead changes the noise distribution and thereby steers the action generated by  $\pi_{\text{BC}}$  without updating the pretrained policy. The objective is  $\max_{\theta} \mathbb{E}_{\pi_{\theta}^w} \sum_{t=0}^T \gamma^t R(\mathbf{s}_t, \mathbf{a}_t)$ . Tactile feedback can be used by  $\pi_{\theta}^w$  to adapt the frozen BC policy toward successful and contact-safe behaviors.

## IV. METHODOLOGY

PEARS combines tactile-conditioned DSRL with physics-guided force reasoning (PFR) to adapt the behavior of a frozen BC policy. DSRL refines position alignment and contact timing, while PFR uses physical priors encoded in a VLM to update a task-specific contact-force goal  $g_t$  based on the interaction history  $\mathbf{h}_t$  after each episode. At each planning step, the actor  $\pi_{\theta}^w$  samples latent noise  $\mathbf{w}_t$  conditioned on  $o_t$ , including visual, proprioceptive, and tactile information. The frozen BC policy maps the noise tensor and its observations to an action chunk. During contact stage, a tactile-gated low-level force controller adjusts the nominal action  $\mathbf{a}_t$  according to  $g_t$  to produce the executed action  $\mathbf{u}_t$ .

The executed action  $\mathbf{u}_t$  is jointly determined by the RL policy  $\pi_{\theta}^w$  and the contact-force goal inferred by PFR. We treat the original environment and the low-level force controller as a single effective environment under partial observability. The force goal  $g_t$  is fixed within each episode ( $g_t = g_e$ ) and updated by PFR between episodes, providing contextual adaptation. We use  $\mathbf{z}_t = (o_t, g_t)$  as a context-augmented observation for the critic  $Q(\mathbf{z}_t, \mathbf{w}_t)$ , while the actor remains conditioned on  $o_t$ . We thus model the interaction as a partially observable Markov decision process (POMDP) and approximate action values using the current context-augmented observation  $\mathbf{z}_t$ .

### A. Real-World Tactile-Informed Latent RL Actor

*a) VLA and VTLA base policies.* We initialize the behavior-cloning policy from GR00T N1.7 [2] and fine-tune it on task demonstrations. Each observation contains an image from the wrist-mounted camera  $\mathbf{I}_t^{\text{wrist}}$  and a third-person image  $\mathbf{I}_t^{\text{ext}}$ , both resized to  $224 \times 224$  pixels. We represent the end-effector pose by its Cartesian position and a continuous 6D orientation representation formed by the first

two rows of its rotation matrix. Together with the current gripper width, the proprioceptive vector is defined as

$$\mathbf{p}_t = [(\mathbf{x}_t^{\text{ee}})^{\top}, \text{vec}(\mathbf{R}_{t,1:2,:}^{\text{ee}})^{\top}, d_t]^{\top} \in \mathbb{R}^{10}. \quad (2)$$

The VLA policy is conditioned on the two images,  $\mathbf{p}_t$ , and the language instruction. For the VTLA policy, we additionally concatenate the 6D force–torque measurements from the left and right fingertips,  $\mathbf{f}_t^L, \mathbf{f}_t^R \in \mathbb{R}^6$ , with  $\mathbf{p}_t$ . Thus, the proprioceptive–tactile state has 22 dimensions.

At each planning step  $t$ , the base policy predicts an action chunk spanning time steps  $t$  through  $t + H - 1$ ,

$$\begin{aligned} \mathbf{A}_t &= [\mathbf{a}_t, \dots, \mathbf{a}_{t+H-1}]^{\top} \in \mathbb{R}^{H \times 10}, \\ \mathbf{a}_{\tau} &= [(\hat{\mathbf{x}}_{\tau}^{\text{ee}})^{\top}, \text{vec}(\hat{\mathbf{R}}_{\tau,1:2,:}^{\text{ee}})^{\top}, \hat{d}_{\tau}]^{\top} \end{aligned} \quad (3)$$

Each row contains the target position, target orientation, and target gripper width. We train the policy with an action horizon  $H = 32$ . During deployment, we use receding-horizon control and execute the first  $K = 10$  actions before acquiring a new observation and replanning.

*b) Tactile-informed latent actor:* Directly processing raw images in the online RL actor would be computationally expensive. We therefore reuse visual tokens from the frozen VLM backbone and pool them into a 2048-dimensional feature  $\bar{\mathbf{v}}_t$ . For the actor,  $o_t$  is represented by this visual feature, the proprioceptive vector  $\mathbf{p}_t$ , and the two fingertip wrenches  $\mathbf{f}_t^L$  and  $\mathbf{f}_t^R$ . After modality-specific linear projections, a Transformer fuses the visual, proprioceptive, and tactile features. Its output head parameterizes a 10-dimensional latent-noise distribution and samples  $\mathbf{w}_t \in \mathbb{R}^{10}$ . GR00T expects a latent-noise tensor in  $\mathbb{R}^{H \times 132}$  for each  $H$ -step action chunk. We repeat the same 10-dimensional actor output across all  $H$  chunk positions, producing an intermediate tensor in  $\mathbb{R}^{H \times 10}$ . At each position, these ten values occupy the first ten channels. The remaining 122 channels retain their corresponding entries from a sample drawn once from a standard Gaussian distribution and held fixed. We include this fixed input expansion in the notation  $\pi_{\text{BC}}(o_t, \mathbf{w}_t)$  for the frozen policy’s output. The critics take  $\mathbf{w}_t$  directly as their action input, so both actor and critic optimization remain in the 10-dimensional noise space.

*c) Actor–critic optimization.* Following DSRL-SAC [7], we treat each executed action chunk as one RL transition. At chunk index  $n$ , the replay buffer stores  $(\mathbf{z}_n, \mathbf{w}_n, r_n, \mathbf{z}_{n+1}, \delta_n)$ , where  $r_n$  is the reward accumulated over the executed chunk and  $\delta_n$  is the terminal flag. We use a standard critic ensemble  $\{Q_{\eta_i}\}_{i \in \mathcal{I}}$  and its corresponding target networks  $\{Q_{\bar{\eta}_i}\}_{i \in \mathcal{I}}$ . For a sampled transition, we draw  $\mathbf{w}_{n+1} \sim \pi_{\theta}^w(\cdot | o_{n+1})$  and form the chunk-level temporal-difference target without entropy backup

$$y_n = r_n + \gamma(1 - \delta_n) \min_{i \in \mathcal{I}} Q_{\bar{\eta}_i}(\mathbf{z}_{n+1}, \mathbf{w}_{n+1}). \quad (4)$$

where  $\gamma$  is the discount factor between successive action chunks. The critics are updated by minimizing

$$\mathcal{L}_Q(\{\eta_i\}_{i \in \mathcal{I}}) = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \mathbb{E}_{\mathcal{B}} \left[ (Q_{\eta_i}(\mathbf{z}_n, \mathbf{w}_n) - y_n)^2 \right]. \quad (5)$$

We then update the actor through reparameterized samples of  $w_n$  while keeping the critics fixed. The actor objective is

$$\mathcal{L}_\pi(\theta) = \mathbb{E}_{\substack{z_n \sim \mathcal{B}, \\ w_n \sim \pi_\theta^w}} \left[ \alpha \log \pi_\theta^w(w_n | o_n) - \min_{i \in \mathcal{I}} Q_{\eta_i}(z_n, w_n) \right], \quad (6)$$

where  $\alpha$  controls the exploration entropy and is adjusted toward a target entropy during training.

### B. PFR-Guided Contact-Force Control

We combine task-specific contact-force bounds inferred by the PFR module with low-level hybrid force–position control. Different manipulation tasks require different forms of force regulation. Fragile-object grasping requires sufficient but non-destructive grasp force, whereas whiteboard wiping requires an appropriate normal force. Fragile fruit harvesting, like raspberry, requires both a gentle grasp and sufficient pulling force to detach the fruit from its stem. This combination provides task-specific force feedback during contact.

We formulate force regulation over contact phases involving gripper–object interaction, object–environment interaction, or both. Let  $\mathbf{f}_t = [\mathbf{F}_t^\top, \boldsymbol{\tau}_t^\top]^\top \in \mathbb{R}^6$  denote the object–environment wrench in the end-effector frame, comprising 3D force and 3D torque. When gripper–object interaction must be constrained, such as to avoid crushing a fragile object, we append the scalar grasp force  $f_t^g$  to obtain  $\bar{\mathbf{f}}_t = [(\mathbf{f}_t)^\top, f_t^g]^\top \in \mathbb{R}^7$ . We project this vector onto the task-relevant force-control axes as

$$\mathbf{f}'_t = \mathbf{A} \bar{\mathbf{f}}_t, \quad \mathbf{A} \in \mathbb{R}^{m \times D}, \quad (7)$$

where each row of  $\mathbf{A}$  defines one regulated force or torque channel and  $m$  is the number of regulated channels. During contact, the projected wrench must satisfy the component-wise constraint

$$\mathbf{f}_{\min} \preceq \mathbf{f}'_t \preceq \mathbf{f}_{\max}. \quad (8)$$

We initialize  $\mathbf{A}$  before RL training using VLM estimates and manual correction. The PFR module infers the task-conditioned bounds  $\mathbf{f}_{\min}$  and  $\mathbf{f}_{\max}$ . The hybrid force–position controller then regulates the selected axes within these bounds during task execution.

*1) Force-Control Axis Estimation :* When object–environment force regulation is required, we constrain interaction along a single task-relevant force or torque axis. Gripper–object interaction is regulated through the scalar grasp force  $f_t^g$  when required. Thus,  $m = 1$  for either object–environment regulation alone or grasp-force regulation alone, and  $m = 2$  when both channels are regulated. Before RL training, we use the VLM to estimate the task-relevant principal force-control axis in the robot’s end-effector coordinate frame. At this stage, we manually review the estimate and correct any erroneous axis. The resulting force or torque axis defines the corresponding row of  $\mathbf{A}$ . When grasp-force regulation is required,  $\mathbf{A}$  includes a row selecting  $f_t^g$ .

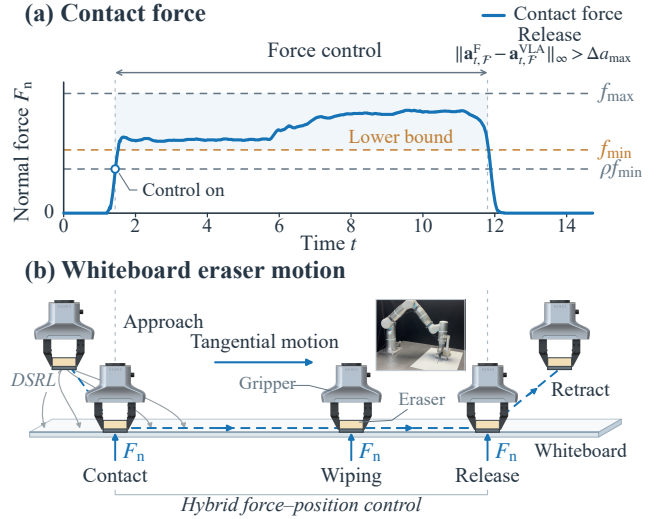


Fig. 2. Stage-aware force control during whiteboard wiping.

*2) PFR Module:* After each episode, PFR queries the VLM with the final contact-phase image, the measured force trajectory, and the force bounds used during execution. The VLM assesses whether the interaction force was insufficient, excessive, or adequate, while distinguishing force-related failures from positioning, trajectory, or force-tracking errors. These diagnoses provide evidence for estimating the task-specific force bounds. The diagnosis determines whether and how the force bounds are updated. An insufficient-force diagnosis shifts the corresponding interval toward a larger force, whereas an excessive-force diagnosis shifts it toward a smaller force. When the evidence is indeterminate, the current estimate is retained. Likewise, if the failure is attributed to positioning or trajectory execution, the force bounds remain unchanged. This outcome-conditioned update has a direct task-dependent interpretation. If visible marks remain on a whiteboard after wiping, PFR increases the admissible normal-force range. If a fragile fruit is crushed during harvesting, it reduces the grasp-force range; if the fruit remains attached to the stem, it instead increases the pulling-force range.

To make PFR decision cumulative rather than myopic, we compress the complete interaction history into the VLM context. The history records the force bounds of each episode, whether the episode succeeded or failed, the diagnosed failure cause, and the resulting update. Consequently, each new estimate is conditioned on both the latest multimodal evidence and the outcomes of all preceding interactions.

*3) Stage-aware Force Control:* Fig. 2 illustrates the framework of contact-stage transitions using the whiteboard wiping task. The low-level force controller runs at 50 Hz, while the policy runs at 30 Hz. Force regulation is activated when the manipulation enters a contact stage, thereby avoiding interference with the policy during free-space motion. For tasks with a single contact stage, let  $c_t^{\text{bi}} \in \{0, 1\}$  indicate that a stable bilateral grasp has been established, and let  $f'_{t,k}$  be the interaction force along the principal control axis  $k$ . The

contact-stage indicator  $\chi_t$  is activated when both the grasp and load conditions are satisfied,

$$\chi_t : 0 \rightarrow 1 \quad \text{if} \quad c_t^{\text{bi}} = 1 \wedge |f'_{t,k}| \geq \rho f_{\min,k}, \quad \rho = 0.7. \quad (9)$$

Once activated, an admittance controller modifies the action coordinates selected by  $\mathbf{A}$ , while the remaining coordinates continue to follow the VLA action. For each regulated channel  $k$ , the force or torque reference is set to the midpoint of the PFR-inferred interval,  $f_k^{\text{ref}} = (f_{\min,k} + f_{\max,k})/2$ .

We also use the VLA output to determine when force control should be released. Let  $\mathbf{a}_{t,\mathcal{F}}^{\text{VLA}}$  and  $\mathbf{a}_{t,\mathcal{F}}^{\text{F}}$  denote, respectively, the VLA command and the force-controller command on the force-controlled coordinates  $\mathcal{F}$ . Their disagreement is the correction required to preserve force control. If this correction exceeds the controller’s admissible range,

$$\|\mathbf{a}_{t,\mathcal{F}}^{\text{F}} - \mathbf{a}_{t,\mathcal{F}}^{\text{VLA}}\|_{\infty} > \Delta a_{\max}, \quad (10)$$

we set  $\chi_{t+1} = 0$  and return control of these coordinates to the VLA. This rule treats a sufficiently large policy command as evidence that the current contact stage has ended, rather than forcing the controller to oppose the intended task motion.

During an active contact stage, the low-level controller regulates the selected axes within the force bounds inferred by PFR. If the measured force reaches the smaller of  $f_{\max,k}$  and the hard safety limit, the controller issues an unloading command. The executed hybrid action is

$$\mathbf{a}_t^{\text{exec}} = \Psi(\mathbf{a}_t^{\text{VLA}}, \mathbf{a}_t^{\text{F}}, \mathbf{A}, \chi_t), \quad (11)$$

where  $\Psi$  decomposes motion in task space into the force-controlled subspace specified by  $\mathbf{A}$  and its complement. When  $\chi_t = 1$ , motion along the selected axes follows  $\mathbf{a}_t^{\text{F}}$ , while motion in the complementary directions follows  $\mathbf{a}_t^{\text{VLA}}$ . When  $\chi_t = 0$ ,  $\Psi$  returns  $\mathbf{a}_t^{\text{VLA}}$ .

For tasks involving multiple contact stages, the VLA policy can be extended to explicitly predict the current contact stage from its observations. This prediction allows the controller to select the appropriate force-control axes and PFR-inferred force bounds, enabling finer force regulation at each stage. For example, during fruit harvesting, the controller can regulate grasp force when establishing gripper–fruit contact and additionally regulate pulling force during fruit–stem detachment, while maintaining a gentle grasp.

### C. The Complete System of PEARS

Algorithm 1 summarizes the complete pipeline, including force-axis initialization, online latent RL, stage-aware force arbitration, and episode-level force adaptation through PFR.

## V. EXPERIMENTS

We organize the experiments around three questions: **Q1: Adaptation efficiency.** Does **PEARS** enable more sample-efficient online adaptation than existing policy fine-tuning methods? **Q2: PFR module.** Can the PFR module in **PEARS** infer task-appropriate contact-force bounds, and how robust are these predictions? **Q3: Real-world deployment.** Can **PEARS** be deployed effectively on physical robotic systems?

### Algorithm 1 PEARS

---

**Require:** Demonstrations  $\mathcal{D}$ ; pretrained VLA  $\pi_{\text{BC}}$ ; VLM  $\mathcal{V}$

- 1: Estimate task-relevant control axes in the end-effector frame
- 2: Manually review and correct the axes; construct  $\mathbf{A}$  from the selected control channels
- 3:  $[\mathbf{f}_{\min}, \mathbf{f}_{\max}] \leftarrow \text{PFRINFERBOUNDS}(\mathcal{V}, \text{task metadata})$
- 4: Initialize  $\pi_{\theta}^w$ , critics, replay buffer  $\mathcal{B}$ , and history  $\mathcal{H}_{\text{epi}}$
- 5: **for**  $e = 1, \dots, E$  **do**
- 6:   Reset the environment;  $t \leftarrow 0$ ;  $\mathcal{T}_e \leftarrow \emptyset$
- 7:   **while** the episode is not terminated **do**
- 8:     Observe  $o_t$  and tactile wrench  $\bar{\mathbf{f}}_t$ ; form  $\mathbf{z}_t = (o_t, g_e)$
- 9:     Sample  $\mathbf{w}_t \sim \pi_{\theta}^w(\cdot | o_t)$
- 10:      $\mathbf{A}_t^{\text{VLA}} \leftarrow \pi_{\text{BC}}(o_t, \mathbf{w}_t)$
- 11:     **for**  $j = t, \dots, t + K - 1$  **do**
- 12:        $\mathbf{a}_j^{\text{VLA}} \leftarrow \mathbf{A}_t^{\text{VLA}}[j - t]$
- 13:        $\mathbf{f}'_j \leftarrow \mathbf{A} \bar{\mathbf{f}}_j$
- 14:        $(\chi_j, k_j) \leftarrow \text{CONTACTSTAGE}(\mathcal{T}_e, \mathbf{f}'_j, \mathbf{a}_j^{\text{VLA}})$
- 15:       **if**  $\chi_j = 1$  **then**
- 16:          $\mathbf{u}_j \leftarrow \text{FORCECONTROL}(\mathbf{a}_j^{\text{VLA}}, \mathbf{f}'_j, k_j)$
- 17:       **else**
- 18:          $\mathbf{u}_j \leftarrow \mathbf{a}_j^{\text{VLA}}$
- 19:       Execute  $\mathbf{u}_j$  and append the transition to  $\mathcal{T}_e$
- 20:       Store the chunk transition with  $\mathbf{w}_t$  in  $\mathcal{B}$
- 21:       Update the critics, latent actor, entropy coefficient, and target critics from  $\mathcal{B}$
- 22:      $t \leftarrow t + K$
- 23:    $\text{PFRUPDATEBOUNDS}(\mathcal{V}, \mathcal{T}_e, \mathcal{H}_{\text{epi}})$

---

### A. Experimental Setup

**1) Evaluation Tasks:** We consider three simulation tasks: Thin Sheet Transfer, Bottle Cap Twisting, and Fragile Fruit Picking. In Thin Sheet Transfer, the robot must transport a fragile thin sheet to the target and regulating its grasp force to avoid crushing it. The object position is randomized within a  $0.2 \text{ m} \times 0.2 \text{ m}$  region, and the maximum allowable interaction force is 1.1–1.6 N. In Bottle Cap Twisting, the robot must tighten a bottle cap and establish an appropriate normal force between the cap and the bottle after tightening. This task evaluates whether the PFR module can infer an appropriate interaction torque. The object position is randomized within a  $0.1 \text{ m} \times 0.1 \text{ m}$  region, and the target torque range for successful completion is 0.35–0.50 N m. In Fragile Fruit Picking, the robot must detach a raspberry from its vine. It must apply a gentle grasp to avoid crushing the fruit while exerting sufficient pulling force for detachment. The object position is randomized by up to  $\pm[8, 10, 12] \text{ mm}$  along the  $x$ -,  $y$ -, and  $z$ -axes, respectively. The crushing-force threshold is 3.2 N, and the minimum force required for detachment is 0.8 N. The preset open-loop pulling offset is zero.

For real-world evaluation, we consider two tasks: Whiteboard Erasing and Pipette Liquid Aspiration. In Whiteboard Erasing, the robot must learn to apply an appropriate contact force between the eraser and the whiteboard to remove the marker traces. In Pipette Liquid Aspiration, the robot must regulate the fingertip contact force on the pipette: the force must be sufficient to lift the pipette while remaining low enough to prevent the liquid inside from being expelled.

For each task, we use 50 offline demonstration trajectories. Base policies are obtained by fine-tuning GR00T N1.7 [2]

TABLE I  
ONLINE ADAPTATION AND FINAL-POLICY PERFORMANCE COMPARISON. “/” DENOTES AN UNREACHED  $E_{80}$ .

| Method          | Thin Sheet Transfer              |             |              |                   | Bottle Cap Twisting              |             |              |                   | Fragile Fruit Picking            |             |              |                   |
|-----------------|----------------------------------|-------------|--------------|-------------------|----------------------------------|-------------|--------------|-------------------|----------------------------------|-------------|--------------|-------------------|
|                 | SR (%)                           | nAUC (%)    | $E_{80}$     | Force failure (%) | SR (%)                           | nAUC (%)    | $E_{80}$     | Force failure (%) | SR (%)                           | nAUC (%)    | $E_{80}$     | Force failure (%) |
|                 | Mean $\pm$ Std. $\uparrow$       | $\uparrow$  | $\downarrow$ | $\downarrow$      | Mean $\pm$ Std. $\uparrow$       | $\uparrow$  | $\downarrow$ | $\downarrow$      | Mean $\pm$ Std. $\uparrow$       | $\uparrow$  | $\downarrow$ | $\downarrow$      |
| Base Policy     | 73.7 $\pm$ 0.6                   | 73.7        | /            | 5.0               | 43.0 $\pm$ 1.0                   | 43.0        | /            | 46.0              | 23.7 $\pm$ 1.2                   | 23.7        | /            | 52.3              |
| Residual RL [6] | 75.7 $\pm$ 0.6                   | 76.0        | 12           | 6.7               | 68.0 $\pm$ 20.3                  | 68.0        | 47           | 20.3              | 37.3 $\pm$ 2.1                   | 23.2        | /            | 44.7              |
| AWR [33]        | 67.7 $\pm$ 2.5                   | 68.2        | 37           | 7.7               | 21.0 $\pm$ 13.1                  | 20.3        | /            | 61.7              | 15.7 $\pm$ 1.2                   | 16.4        | /            | 55.7              |
| DPPO [34]       | 4.7 $\pm$ 3.5                    | 2.5         | /            | 5.7               | 32.7 $\pm$ 1.5                   | 31.7        | /            | 55.3              | 22.3 $\pm$ 2.3                   | 16.3        | /            | 48.0              |
| DSRL [7]        | 82.3 $\pm$ 4.5                   | 72.2        | 29           | 9.7               | 57.7 $\pm$ 1.5                   | 57.9        | /            | 31.3              | 40.3 $\pm$ 4.2                   | 25.3        | /            | 36.3              |
| <b>PEARS</b>    | <b>94.7 <math>\pm</math> 1.5</b> | <b>94.7</b> | <b>10</b>    | <b>2.0</b>        | <b>87.7 <math>\pm</math> 1.5</b> | <b>90.1</b> | <b>22</b>    | <b>10.3</b>       | <b>77.7 <math>\pm</math> 1.2</b> | <b>73.4</b> | <b>25</b>    | <b>3.3</b>        |

on the corresponding offline demonstrations. For Thin Sheet Transfer and real-world tasks, the base policies do not use tactile features. For Bottle Cap Twisting and Fragile Fruit Picking, the state additionally includes the 6D force-torque measurements from both fingertips. All simulation experiments are conducted in NVIDIA Isaac Sim.

2) *Training and Evaluation Detail*: The GR00T-based policies are trained on an NVIDIA H100 GPU for a fixed 30 epochs, and the checkpoint from the final epoch is used as the base policy. Training takes approximately five hours per task. For the baseline comparisons, simulation-based RL training is performed on an NVIDIA GeForce RTX 5090 GPU, with GPT-5.6 Sol configured at extra-high reasoning effort as the VLM backbone of PFR. For the real-world experiments, policy inference and online RL training are also performed on an NVIDIA GeForce RTX 5090 GPU.

3) *Baselines*: We compare **PEARS** with a **frozen base policy** and four representative online policy-adaptation methods. **Residual RL** [6], [35] uses a lightweight residual network to correct the actions generated by the frozen base policy, whereas **DSRL** [7] learns a tactile-conditioned actor in the policy’s latent noise space. **AWR** [33] updates the policy through advantage-weighted regression, and **DPPO** [34] uses a PPO-style objective to update the diffusion policy. **PEARS**, **Residual RL**, and **DSRL** combine a tactile-conditioned actor. For all RL algorithms, we use a sparse terminal reward of +10 for a successful episode and −10 for a failed episode, with a discount factor of  $\gamma = 0.9$ . In simulation, success or failure is determined automatically by the simulator, whereas real-world outcomes are labeled manually by a human operator.

### B. Comparison with Baselines (Q1)

Each adaptive method used three 100-episode online runs, followed by 100 test trials per run. Final-policy success rate (SR) is reported as mean  $\pm$  sample SD. Let  $SR_e \in [0, 1]$  denote trailing-10 online success averaged across runs (Fig. 3). Normalized area under the learning curve (nAUC) is the mean of  $SR_e$  for  $e = 10, \dots, 100$ , expressed as a percentage;  $E_{80}$  is the first episode with  $SR_e \geq 0.8$  (“/” if unreachable). nAUC measures overall online performance, while a smaller  $E_{80}$  indicates faster attainment of a high success rate. Force-related failure rate is the percentage of all episodes ending in task-specific force or contact failures,

pooled across runs.

PEARS achieved the highest SR on Thin Sheet Transfer, Bottle Cap Twisting, and Fragile Fruit Picking: 94.7%, 87.7%, and 77.7% (Table I). These exceed the strongest per-task baselines by 12.4, 19.7, and 37.4 percentage points. It also led in nAUC (94.7%, 90.1%, and 73.4%), indicating higher success throughout online adaptation. The largest SR gain occurs on Fragile Fruit Picking, which requires coordinated grasping and pulling without damaging the fruit.

PEARS reached  $E_{80}$  at episodes 10, 22, and 25. On Thin Sheet Transfer, episode 10 is the earliest evaluable window, compared with 12 for the fastest baseline, Residual RL. On Bottle Cap Twisting, Residual RL required 47 episodes, so PEARS reduced interactions to the threshold by 53.2%. No baseline reached 80% on Fragile Fruit Picking within 100 episodes. Higher success was accompanied by the lowest force-related failure rates (2.0%, 10.3%, and 3.3%), compared with 9.7%, 20.3%, and 36.3% for the strongest SR baselines. The mean waiting time for a VLM response was 26.8 s, which is acceptable because this wait can overlap with other steps, such as environment resets.

### C. Evaluation of the PFR Module (Q2)

Fig. 4 shows how PFR corrects initially inaccurate force and torque intervals in two tasks. Over the first 12 episodes, predicted force and torque intervals stabilize within ground-truth ranges, illustrating refinement of initial estimates from interaction outcomes.

Table II compares five variants. *DSRL only* omits PFR and external force control; *w/o DSRL* combines PFR and force control with the frozen base policy. *Fixed Range* and *Rand Search* retain DSRL and force control, respectively holding the first PFR-inferred intervals fixed or randomly searching intervals online. For *Rand Search*, the checkpoint with the highest mean online success is used for evaluation. *Qwen3* replaces GPT-5.6 Sol with Qwen3-VL-2B [36] in PFR.

On Thin Sheet Transfer, DSRL alone achieved 82.3% SR, compared with 77.0% for PFR-guided force control without DSRL. Combining both increased SR to 94.7%, versus 84.7% with fixed intervals. On Bottle Cap Twisting, PFR-guided force control without DSRL retained 84.7% SR, whereas DSRL alone achieved 57.7% and *Fixed Range* 62.3%. The full method reached 87.7%, indicating a greater benefit from adapting interaction torque. The different ab-

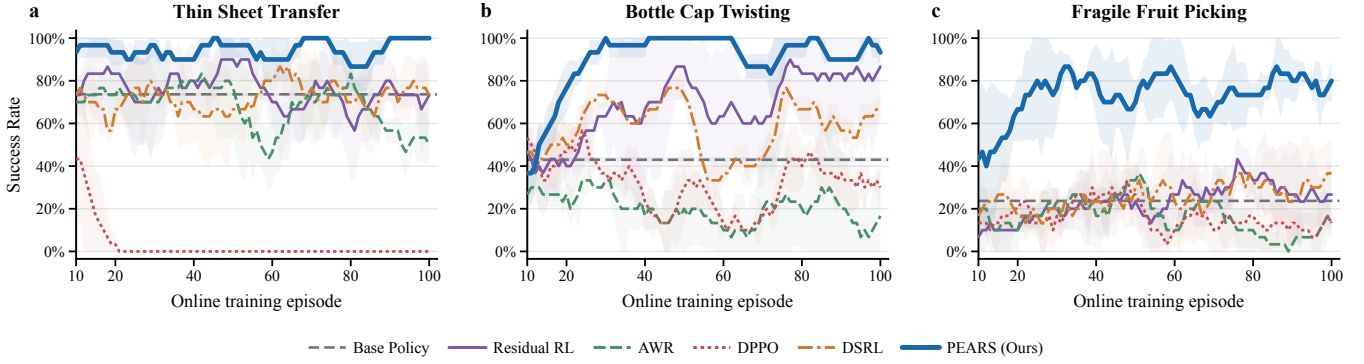


Fig. 3. Online adaptation success across three simulation tasks. Curves show mean trailing-10 success rates across three 100-episode runs, using complete windows ending at episodes 10–100. Shaded bands denote mean  $\pm$  one sample standard deviation (SD)

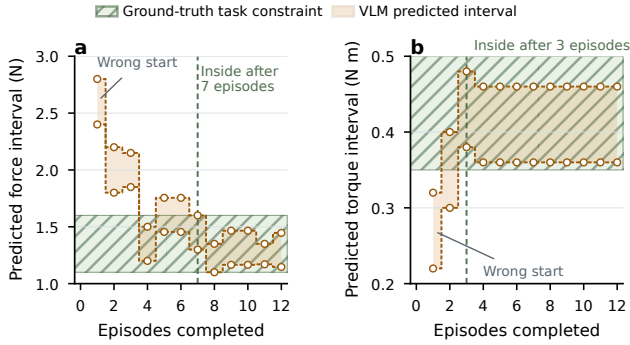


Fig. 4. Example of force and torque intervals predicted by PFR during online interaction. a) Thin Sheet Transfer. b) Bottle Cap Twisting.

TABLE II

ABLATION OF PFR, DSRL, FORCE-RANGE SELECTION, AND THE VLM BACKBONE USED IN PFR ACROSS THREE SIMULATION TASKS.

| SR (%) | DSRL only      | w/o DSRL       | Fixed Range    | Rand Search    | Qwen3          | PEARS                            |
|--------|----------------|----------------|----------------|----------------|----------------|----------------------------------|
| Sheet  | 82.3 $\pm$ 4.5 | 77.0 $\pm$ 1.0 | 84.7 $\pm$ 3.8 | 88.3 $\pm$ 6.0 | 93.0 $\pm$ 1.7 | <b>94.7 <math>\pm</math> 1.5</b> |
| Cap    | 57.7 $\pm$ 1.5 | 84.7 $\pm$ 1.5 | 62.3 $\pm$ 2.5 | 77.0 $\pm$ 5.6 | 86.7 $\pm$ 0.6 | <b>87.7 <math>\pm</math> 1.5</b> |
| Fruit  | 40.3 $\pm$ 4.2 | 61.7 $\pm$ 2.1 | 44.0 $\pm$ 2.6 | 40.3 $\pm$ 2.5 | 67.0 $\pm$ 2.6 | <b>77.7 <math>\pm</math> 1.2</b> |

lation rankings suggest that motion adaptation contributes more to Thin Sheet Transfer, whereas force control is more important for Bottle Cap Twisting. On Fragile Fruit Picking, removing PFR and force control or removing DSRL reduced SR from 77.7% to 40.3% or 61.7%, respectively; fixed intervals yielded 44.0%. Fixed intervals provide only modest gains over DSRL across tasks, suggesting that online correction is important when initial bounds are inaccurate. These results support complementary roles for learned motion adaptation and online force-interval correction.

*Rand Search* increased SR over DSRL on the two single-interval tasks, but remained below PEARS. Random search can find partially suitable bounds in these tasks. For example, in one Thin Sheet Transfer run, its interval [1.32, 1.98] N partially overlaps the ground-truth range [1.1, 1.6] N. This permits useful forces, while the excessive upper bound helps explain the remaining performance gap. On two-stage Fragile

TABLE III

REAL-WORLD PERFORMANCE (SUCCESSFUL TRIALS / TOTAL TRIALS).

| Task                      | Base Policy | DSRL [7] | PEARS   |
|---------------------------|-------------|----------|---------|
| Whiteboard Erasing        | 11 / 20     | 15 / 20  | 19 / 20 |
| Pipette Liquid Aspiration | 8 / 20      | 12 / 20  | 18 / 20 |

Fruit Picking, its 40.3% mean SR matches DSRL, consistent with the difficulty of randomly identifying suitable grasping and pulling intervals jointly. A usable grasping interval alone cannot ensure success if pulling remains ineffective, so useful updates must account for both contact stages. The difference is also evident in adaptation speed. On Thin Sheet Transfer and Bottle Cap Twisting, PEARS reached  $E_{80} = 10$  and 22, versus 55 and 83 for *Rand Search*. Earlier attainment of 80% mean online success suggests that VLM reasoning provides useful direction for interval updates and improves online sample efficiency.

Qwen3 achieved SRs of 93.0%, 86.7%, and 67.0%, trailing PEARS by 1.7, 1.0, and 10.7 percentage points, respectively. Both VLM backbones improve over DSRL alone across tasks, indicating that the lightweight model still provides useful force guidance. The larger gap on Fragile Fruit Picking suggests that reasoning quality matters more when inferring force bounds with multiple stages.

#### D. Real-world Experiment (Q3)

For each real-world task, DSRL and PEARS are allocated 40 episodes of online interaction. We then evaluate each adapted policy and the frozen base policy over 20 test trials (Table III). In Whiteboard Erasing, sufficient normal force is needed to remove marker traces, whereas excessive force can dislodge the eraser from the gripper. The base policy succeeds in 11/20 trials; its failures comprise three wiping misalignments, two contact-timing errors that leave residual marks, and four cases of insufficient force. DSRL improves success to 15/20, with two misalignment failures and three insufficient-force failures. PEARS reaches 19/20, with the sole failure caused by misalignment. In Pipette Liquid Aspiration, any liquid leakage during transfer counts as a failure. The base policy succeeds in 8/20 trials, with

four initial grasping misalignments, six cases of leakage during transfer, and two failures during final dispensing. DSRL reaches 12/20, with two grasping misalignments, five cases of liquid expulsion during transfer, and one dispensing failure. PEARS succeeds in 18/20, with both failures caused by initial grasping misalignment.

These results suggest that DSRL alone struggles to learn precise force regulation within 40 episodes, particularly for liquid transfer. The absence of force-related failures with PEARS in these tests supports the benefit of using PFR to adapt force bounds from tactile feedback and enforcing them through low-level force control.

## VI. CONCLUSION & FUTURE WORK

We presented PEARS, a framework for rapid post-training of pretrained policies that combines PFR with traditional DSRL. PFR uses tactile feedback and visual outcomes to diagnose failure causes and adapt contact-force bounds, which are enforced by a low-level controller, while DSRL refines the behavior of the frozen base policy. Across three simulation tasks, PEARS improved success rates by 12.4–37.4 percentage points over the strongest baseline for each task, with faster online adaptation and fewer force-related failures. In real-world tasks, PEARS achieved 19/20 and 18/20 successful test trials on Whiteboard Erasing and Pipette Liquid Aspiration, respectively.

Currently, PFR provides corrective guidance only for failures caused by inappropriate force; other errors still rely on conventional RL for correction. Future work will explore integrating failure reasoning more broadly into RL to guide exploration and policy updates beyond force regulation. This could reduce costly trial-and-error interaction and improve the efficiency of learning on physical robots.

## REFERENCES

- [1] P. Intelligence, *et al.*, “ $\pi_{0.5}$ : a vision-language-action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025.
- [2] NVIDIA, *et al.*, “GR00T N1: An open foundation model for generalist humanoid robots,” in *ArXiv Preprint*, March 2025.
- [3] C. Yuan, *et al.*, “Ftp-1: A generalist foundation tactile policy across tactile sensors for contact-rich manipulation,” *arXiv preprint arXiv:2606.13102*, 2026.
- [4] C. Parés-Morlans, N. Kuhn, I. Liu, A. Longhini, and J. Bohg, “Demystifying when and why vlas fail in contact-rich tasks and how to fix them,” *arXiv preprint arXiv:2608.01402*, 2026.
- [5] S. Ma, *et al.*, “Taccorl: Integrating tactile feedback into vla via simulation,” *arXiv preprint arXiv:2606.11743*, 2026.
- [6] K. Yu, H. Zhang, H. Ravichandar, Y. Han, and R. Gao, “OmniTacTune: Policy-agnostic real-world RL for tactile residual adaptation of visual policies,” *arXiv preprint arXiv:2607.03723*, 2026.
- [7] A. Wagenmaker, *et al.*, “Steering your diffusion policy with latent space reinforcement learning,” in *Proceedings of the 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 2025, pp. 258–282.
- [8] J. Zhao, N. Kuppuswamy, S. Feng, B. Burchfiel, and E. H. Adelson, “PolyTouch: A robust multi-modal tactile sensor for contact-rich manipulation using tactile-diffusion policies,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2025, pp. 104–110.
- [9] C. Higuera, *et al.*, “Sparsht: Self-supervised touch representations for vision-based tactile sensing,” in *Proceedings of the 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2025, pp. 885–915.
- [10] B. Chen, *et al.*, “Univtac: A unified simulation platform for visuo-tactile manipulation data generation, learning, and benchmarking,” *arXiv preprint arXiv:2602.10093*, 2026.
- [11] C. Xu, *et al.*, “VT-MUSE: Multimodal unified sequential visuo-tactile representation learning for manipulation,” *arXiv preprint arXiv:2608.21290*, 2026.
- [12] J. Jones, O. Mees, C. Sferrazza, K. Stachowicz, P. Abbeel, and S. Levine, “Beyond sight: Finetuning generalist robot policies with heterogeneous sensors via language grounding,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2025, pp. 5961–5968.
- [13] Z. Zhang, *et al.*, “TouchGuide: Inference-time steering of visuomotor policies via touch guidance,” in *Proceedings of Robotics: Science and Systems*, Sydney, Australia, July 2026.
- [14] Y. Huang, *et al.*, “TaF-VLA: Tactile-force alignment in vision-language-action models for force-aware manipulation,” *arXiv preprint arXiv:2601.20321*, 2026.
- [15] L. Heng, H. Geng, K. Zhang, P. Abbeel, and J. Malik, “ViTacFormer: Learning cross-modal representation for visuo-tactile dexterous manipulation,” in *Proceedings of Robotics: Science and Systems*, Sydney, Australia, July 2026.
- [16] Y. Zheng, *et al.*, “OmniVTA: Visuo-tactile world modeling for contact-rich robotic manipulation,” *arXiv preprint arXiv:2603.19201*, 2026.
- [17] N. Team and F. T. Team, “ $n_0$ -vlla: Scaling vision-tactile-language-action model with latent tactile tokens,” 2026.
- [18] D. Niu, *et al.*, “T-rer: Tactile-reactive dexterous manipulation,” *arXiv preprint arXiv:2606.17055*, 2026.
- [19] Z. He, H. Fang, J. Chen, H.-S. Fang, and C. Lu, “FoAR: Force-aware reactive policy for contact-rich robotic manipulation,” *IEEE Robotics and Automation Letters*, vol. 10, no. 6, pp. 5625–5632, 2025.
- [20] H. Xue, *et al.*, “Reactive diffusion policy: Slow-fast visual-tactile policy learning for contact-rich manipulation,” in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025.
- [21] H. Fang, *et al.*, “Force policy: Learning hybrid force-position control policy under interaction frame for contact-rich manipulation,” in *Proceedings of Robotics: Science and Systems*, Sydney, Australia, July 2026.
- [22] Y. Li, *et al.*, “ForceVLA2: Unleashing hybrid force-position control with force awareness for contact-rich manipulation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, June 2026.
- [23] J. Bi, K. Y. Ma, C. Hao, M. S. Zheng, and H. Soh, “VLA-Touch: Enhancing vision-language-action model with dual-level tactile feedback,” *IEEE Robotics and Automation Letters*, vol. 11, no. 7, pp. 8487–8494, 2026.
- [24] X. Li, *et al.*, “AT-VLA: Adaptive tactile injection for enhanced feedback reaction in vision-language-action models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, June 2026.
- [25] Y. Li, *et al.*, “Favla: A force-adaptive fast-slow vla model for contact-rich robotic manipulation,” *arXiv preprint arXiv:2602.23648*, 2026.
- [26] X.-D. Zhang, *et al.*, “Unitacvla: Unified tactile understanding and prediction in vision language action models,” *arXiv preprint arXiv:2606.31723*, 2026.
- [27] G. Lu, *et al.*, “VLA-RL: Towards masterful and general robotic manipulation with scalable reinforcement learning,” *arXiv preprint arXiv:2505.18719*, May 2025.
- [28] H. Li, *et al.*, “SimpleVLA-RL: Scaling VLA training via reinforcement learning,” in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=TQhSodCM4r>
- [29] A. Amin, *et al.*, “ $\pi_{0.6}^*$ : A VLA that learns from experience,” in *Proceedings of Robotics: Science and Systems*, Sydney, Australia, July 2026.
- [30] C. Xu, *et al.*, “RI token: Bootstrapping online rl with vision-language-action models,” *arXiv preprint arXiv:2604.23073*, 2026.
- [31] H. Zheng, *et al.*, “Tori-vla: Tactile guided online reinforcement learning for contact-rich manipulation,” *arXiv preprint arXiv:2606.09337*, 2026.
- [32] K. Black, *et al.*, “ $\pi_0$ : A vision-language-action flow model for general robot control,” in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025.
- [33] X. B. Peng, A. Kumar, G. Zhang, and S. Levine, “Advantage-weighted regression: Simple and scalable off-policy reinforcement learning,” *arXiv preprint arXiv:1910.00177*, 2019.

- [34] A. Ren, *et al.*, “Diffusion policy policy optimization,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 77 288–77 329.
- [35] L. Ankile, Z. Jiang, R. Duan, G. Shi, P. Abbeel, and A. Nagabandi, “Residual off-policy rl for finetuning behavior cloning policies,” *arXiv preprint arXiv:2509.19301*, 2025.
- [36] S. Bai, *et al.*, “Qwen3-v1 technical report,” *arXiv preprint arXiv:2511.21631*, 2025.